



Agentes: crenças

Prof. Elder Santos



Crenças

Fatos

O que o agente considera como Verdade

Regras

Relações entre fatos, uma consulta ou inferência



Fatos

tenhoFome.

acordado(**marvin**) . #marvin é uma constante - 1a letra minúscula

acordado(thanos) .

noCelular(thanos) .

estouEm(**0,1**) . #números também são constantes

comendo(marvin, **Algo**) #Algo é uma variável, pode ser qualquer coisa



Fatos (negação)

`acordado(marvin) .`

`~acordado(hugo) . # hugo não está acordado`

`~noCelular(marvin) . # marvin não está no celular`

`~ → negação explícita`



Regras

`acordado(marvin) .`

`~acordado(hugo) .`

`~noCelular(marvin) .`

`acordado(thanos) .`

`noCelular(thanos) .`

`alunoAtento( Al ) :- acordado( Al )` #Al é uma variável

→ `alunoAtento`, se `acordado`

→ se Al está acordado, então Al está atento



Regras (uso do not)

`acordado(marvin) .`

`~acordado(hugo) .`

`~noCelular(marvin) .`

`acordado(thanos) .`

`noCelular(thanos) .`

`alunoAtento(Al) :- acordado(Al) & not noCelular(Al) .`

→ not: “não acredita que” (não pode ser comprovado pelas crenças)



Regras - Unificação de Variáveis

```
gostoDe(banana, queijo).  
possoUsar(Ing) :- gostoDe(Ing).
```

```
possoUsar(banana) X
```

```
gostoDe(banana, queijo).  
possoUsar(Ing) :- gostoDe(Ing, Qualquer). #variáveis podem ser  
usadas para a unificação/match entre regra e fato
```

```
possoUsar(banana) ok
```



Regras - Unificação de Variáveis

```
gostoDe(banana, queijo).
```

```
gostoDe(Tudo).
```

```
possoUsar(Ing) :- gostoDe(Ing).
```

```
possoUsar(banana) ok
```

```
possoUsar(detergente) ok
```




Regras - Unificação de Variáveis

```
salgado(queijo).
```

```
gostoDe(banana, queijo).
```

```
misturoBananaComSalgado :- gostoDe(banana, I) & salgado(I).
```

```
# quando I tem um valor, esse valor pode ser usado à  
vontade (para outros testes/unificações)
```

Regras - Expressões

`estouEm(2,2).`

`pertoDe(X,Y) :- estouEm(X + 2,Y) | estouEm(X - 2, Y) |
estouEm(X,Y + 2) | estouEm(X, Y - 2) |`

`pertoDe(X,Y) :- L1 = X + 2 & L2 = X - 2 & L3 = Y + 2 ...
(estouEm(L1, Y) | estouEm(L2, Y) |
(estouEm(X, Y) | estouEm(L3, Y))).`

`dentroLimite(X) :- X >= 0 & X <= 10.`